

PowerPredict - Dokumentacija

Autori: Luka Zelembaba, Dušan Đurić

Datum: jun 2026

1. Uvod

PowerPredict je studentski projekat za predikciju satne potrošnje električne energije domaćinstva na osnovu vremenske serije potrošnje. Projekat je prvobitno obuhvatao preprocessing, feature engineering, baseline model i rekurentne neuronske mreže GRU i LSTM, a zatim je nadograđen klasičnim tree-based regresionim modelima: Random Forest Regressor i Gradient Boosting Regressor.

Cilj projekta je poređenje više pristupa za isti regresioni problem:

- jednostavan baseline model zasnovan na vrednosti iz istog sata prethodnog dana,
- sekvencijalni neuronski modeli GRU i LSTM,
- tabularni tree modeli Random Forest i Gradient Boosting.

Svi modeli se porede nad istim validation i test skupovima, koriste iste evaluacione metrike i čuvaju rezultate u standardizovanom obliku: CSV izveštaje, predikcije, sačuvane modele i grafičke prikaze.

2. Tehnički Aspekti Projekta

Projekat je implementiran u programskom jeziku **Python**. Kod je organizovan kao sekvencijalni pipeline kroz fajlove `src/step01_...` do `src/step11_...`. Osnovni workflow pokreće se iz `main.py`, dok se zahtevniji delovi treninga pokreću posebnim komandama.

Za upravljanje okruženjem koristi se **uv**. Zavisnosti projekta nalaze se u `pyproject.toml`, `uv.lock` čuva zaključane verzije paketa radi ponovljivosti okruženja.

Glavne korišćene biblioteke su:

- **pandas** - učitavanje CSV fajlova, obrada tabela, resamplovanje i transformacija vremenskih serija,
- **numpy** - numeričke operacije i računanje metrika,
- **matplotlib** - generisanje grafičkih prikaza rezultata,
- **scikit-learn** - Random Forest Regressor i Gradient Boosting Regressor,
- **PyTorch** - GRU i LSTM neuronski modeli,
- **tqdm** - prikaz napretka tokom dužih treninga,
- **argparse** - izbor pipeline-a kroz komandnu liniju.

Rezultati obrade, modeli, predikcije, log fajlovi i grafici čuvaju se u folderima:

```
data/logs/  
data/predictions/  
data/graphs/  
models/
```

3. Struktura Pipeline-a

Pipeline je podeljen u sledeće korake:

Korak	Fajl	Opis
01	src/step01_preprocessing.py	Čišćenje sirovih podataka i resamplovanje na satni nivo
02	src/step02_features.py	Kreiranje target, vremenskih, lag, rolling i holiday feature-a
03	src/step03_split.py	Vremenska podela na train, validation i test skupove
04	src/step04_baseline_model.py	Baseline model pomoću lag_24h
05	src/step05_baseline_visualization.py	Baseline grafici
06	src/step06_gru_model.py	Treniranje GRU modela
07	src/step07_gru_visualization.py	GRU grafici i baseline poređenje
08	src/step08_lstm_model.py	Treniranje LSTM modela
09	src/step09_lstm_visualization.py	LSTM grafici i baseline poređenje
10	src/step10_tree_models.py	Random Forest i Gradient Boosting trening
11	src/step11_tree_visualization.py	Tree grafici i poređenje svih modela

Glavne komande za pokretanje:

```
uv run main.py          # osnovni workflow i baseline grafike
uv run main.py gru-pipeline # GRU trening, izveštaji, predikcije i grafike
uv run main.py lstm-pipeline # LSTM trening, izveštaji, predikcije i grafike
uv run main.py tree-pipeline # RF/GB trening, izveštaji, predikcije i grafike
```

3.1. Pokretanje Pipeline-a

Prvo se pokreće osnovni pipeline:

```
uv run main.py
```

Ova komanda izvršava preprocessing, feature engineering, vremensku podelu na train/validation/test skupove, baseline model i baseline grafike. Nakon ovog koraka treba da postoje:

```
data/processed/split/train.csv
data/processed/split/validation.csv
data/processed/split/test.csv
data/logs/baseline_report.csv
data/graphs/baseline_graphs/
```

GRU i LSTM pipeline-i pokreću se odvojeno zato što treniraju veći broj PyTorch modela kroz grid search i mogu trajati znatno duže, posebno ako se izvršavaju na CPU-u. Ako je dostupna CUDA/GPU podrška, PyTorch je može koristiti za ubrzavanje treninga.

Provera PyTorch instalacije može se uraditi komandom:

```
uv run python -c "import torch; print(torch.__version__); print(torch.cuda.is_available())"
```

GRU pipeline se pokreće komandom:

```
uv run main.py gru-pipeline
```

Ova komanda trenira sve validne GRU konfiguracije, čuva `.pt` modele, predikcije, training history fajlove, CSV izveštaje i grafike za GRU model, kao i poređenje najboljeg GRU modela sa baseline modelom.

LSTM pipeline se pokreće komandom:

```
uv run main.py lstm-pipeline
```

Ova komanda radi isti tip obrade kao GRU pipeline, ali za LSTM arhitekturu.

Tree pipeline se pokreće komandom:

```
uv run main.py tree-pipeline
```

Ova komanda trenira Random Forest i Gradient Boosting konfiguracije iz `src/step10_tree_models.py`, čuva `.pkl` modele, predikcije, izveštaje, feature importance tabele i grafike. Nakon treninga se generišu i uporedni grafici za sve najbolje modele: baseline, GRU, LSTM, Random Forest i Gradient Boosting.

4. Dataset

Korišćen je **Individual Household Electric Power Consumption dataset**. Dataset sadrži merenja električne potrošnje jednog domaćinstva u Sceaux-u, u Francuskoj, na minutnom nivou.

Izvori:

- [UCI Machine Learning Repository](#), takođe dostupan na:
- [Kaggle dataset](#)

Originalni podaci sadrže minutna merenja potrošnje i pratećih električnih veličina. Za potrebe projekta podaci su prebačeni na satni nivo, jer je cilj predikcija satne potrošnje energije.

Kratak izveštaj o dataset-u čuva se u:

```
data/logs/dataset_report.csv
```

5. Preprocessing

Preprocessing se nalazi u `src/step01_preprocessing.py`.

Glavni koraci su:

- učitavanje raw dataset-a uz tretiranje znaka `?` kao nedostajuće vrednosti,
- spajanje `Date` i `Time` kolona u jedinstvenu `datetime` kolonu,
- provera nekompletnih dana i redova sa nedostajućim numeričkim vrednostima,
- uklanjanje problematičnih dana ili pojedinačnih redova,
- resamplovanje minutnih podataka na satni nivo,
- čuvanje obrađenog dataset-a.

Pošto su raw podaci minutni, za jedan kompletan dan očekuje se **1440 redova**. Ako danu fali više od 60 redova ili ima više od 60 redova sa nedostajućim numeričkim vrednostima, briše se ceo dan. Ako je takvih redova 60 ili manje, dan se zadržava, a uklanjaju se samo pojedinačni redovi sa nedostajućim vrednostima.

Nakon preprocessinga zadržavaju se relevantne kolone:

- `datetime`,
- `Global_active_power`,

- `Global_reactive_power` ,
- `Global_intensity` .

Detalji preprocessinga čuvaju se u:

```
data/logs/preprocessing_report.csv
data/processed/preprocessed.csv
```

6. Feature Engineering

Feature engineering se nalazi u `src/step02_features.py` .

Target kolona je:

```
Energy_kWh
```

Pošto je dataset resamplovan na satni nivo, vrednost `Energy_kWh` numerički odgovara koloni `Global_active_power` za taj sat. Target se uvodi kao posebna kolona da bi ciljna promenljiva bila jasno odvojena od ulaznih feature-a.

Nakon feature engineering-a dobijaju se sledeće kolone:

Kolona	Opis
<code>datetime</code>	vremenska oznaka sata
<code>Global_active_power</code>	aktivna snaga, numerički ekvivalent targeta
<code>Global_reactive_power</code>	reaktivna snaga u trenutnom satu
<code>Global_intensity</code>	intenzitet struje u trenutnom satu
<code>Energy_kWh</code>	target promenljiva
<code>hour</code>	sat u danu
<code>day_of_week</code>	dan u nedelji, gde je 0 ponedeljak
<code>month</code>	mesec u godini
<code>is_weekend</code>	indikator vikenda
<code>is_night</code>	indikator noćnih sati
<code>lag_1h</code>	potrošnja iz prethodnog sata
<code>lag_24h</code>	potrošnja iz istog sata prethodnog dana
<code>lag_168h</code>	potrošnja iz istog sata prethodne nedelje
<code>rolling_mean_24h</code>	prosek potrošnje u prethodna 24 sata
<code>rolling_std_7d</code>	standardna devijacija potrošnje u prethodnih 7 dana
<code>is_holiday</code>	indikator praznika u Francuskoj

Posebno je važno da se rolling feature-i računaju nad shiftovanom target kolonom. To znači da se za trenutni sat ne koristi vrednost targeta iz tog istog sata, već samo prethodno poznate vrednosti. Time se sprečava data leakage. Nakon kreiranja lag i rolling kolona uklanjaju se početni redovi za koje nema dovoljno istorije.

Output:

```
data/processed/features.csv
```

Kolona `Energy_kWh` ostaje u `features.csv`, jer predstavlja target koji modeli treba da predvide. Međutim, način na koji se ta kolona koristi razlikuje se po tipu modela:

- kod GRU/LSTM modela sekvenca sadrži samo prethodne sate, pa model može koristiti istorijske vrednosti `Energy_kWh`, ali ne i `Energy_kWh` iz sata koji se predviđa,
- kod Random Forest i Gradient Boosting modela ulaz je jedan red za trenutni sat, pa se `Energy_kWh` mora eksplicitno izbaciti iz ulaznih feature-a.

Isto važi i za ostala merenja iz trenutnog sata: kod sekvencijalnih modela ona su dostupna samo za prethodne sate u sekvenci, dok bi kod tree modela predstavljala direktna merenja trenutnog sata i zato se izbacuju.

7. Train, Validation i Test Podela

Podela se nalazi u `src/step03_split.py`.

Podaci se dele po vremenskoj osi, bez nasumičnog mešanja redova. Ovakav pristup je neophodan za vremenske serije, jer model u realnoj primeni ne sme da uči iz budućih vrednosti.

Podela:

Skup	Period	Udeo
Train	2006-12-24 do 2009-09-14	približno 70%
Validation	2009-09-14 do 2010-04-18	približno 15%
Test	2010-04-18 do 2010-11-25	približno 15%

Output:

```
data/processed/split/train.csv
data/processed/split/validation.csv
data/processed/split/test.csv
data/logs/split_report.csv
```

8. Evaluacione Metrike

Za sve modele koriste se iste metrike:

Metrika	Formula	Značenje
MAE	$(1 / n) * \sum \text{abs}(y_i - \hat{y}_i)$	prosečna apsolutna greška u kWh
RMSE	$\sqrt{(1 / n) * \sum (y_i - \hat{y}_i)^2}$	jače kažnjava velike greške
MAPE	$(100 / n) * \sum \text{abs}((y_i - \hat{y}_i) / y_i)$	procentualna greška
sMAPE	$(100 / n) * \sum (2 * \text{abs}(y_i - \hat{y}_i) / (\text{abs}(y_i) + \text{abs}(\hat{y}_i)))$	stabilnija procentualna greška
WAPE	$100 * (\sum \text{abs}(y_i - \hat{y}_i) / \sum \text{abs}(y_i))$	ukupna greška u odnosu na ukupnu potrošnju

Glavna metrika za izbor najboljih konfiguracija je **validation MAE** zbog lakog intuitivnog razumevanja. Test skup se koristi tek nakon izbora modela, za finalnu evaluaciju.

9. Baseline Model

Baseline model se nalazi u `src/step04_baseline_model.py`.

Koristi se jednostavno pravilo:

```
predikcija za trenutni sat = potrošnja iz istog sata prethodnog dana
```

Odnosno koristi se feature:

```
lag_24h
```

Baseline je važan jer predstavlja referentnu tačku. Napredniji modeli imaju smisla samo ako daju bolje rezultate od ovog jednostavnog pristupa.

Rezultati:

Skup	MAE	RMSE	MAPE	sMAPE	WAPE
Validation	0.6624	0.9550	74.7894	52.0735	53.0450
Test	0.5123	0.7611	66.8003	49.2873	53.4677

Output:

```
data/logs/baseline_report.csv
data/graphs/baseline_graphs/
```

Baseline grafici obuhvataju prikaz grešaka po satima, scatter odnos stvarnih i predviđenih vrednosti i distribuciju apsolutne greške.

10. GRU Model

GRU (Gated Recurrent Unit) model se nalazi u `src/step06_gru_model.py`.

Koristi se za modelovanje vremenskih serija. Za razliku od baseline modela, GRU ne koristi samo jednu prethodnu vrednost, već dobija sekvencu prethodnih sati.

Za jednu predikciju model dobija ulaz oblika:

```
batch_size x sequence_length x broj_featurea
```

Za svaki target sat formira se sekvenca prethodnih `sequence_length` sati. U funkciji za kreiranje sekvenci koristi se prozor koji se završava pre target sata, pa model ne dobija vrednosti iz trenutnog sata koji predviđa.

Iz liste feature kolona za GRU izbacuju se:

- `datetime`,
- `Global_active_power`.

`datetime` se izbacuje zato što nije direktan numerički ulaz u mrežu. `Global_active_power` se izbacuje zato što je numerički ekvivalent targeta `Energy_kWh`.

Kolona `Energy_kWh` ostaje dostupna u sekvencijalnom ulazu samo za prethodne sate, zato što sekvenca ne uključuje target sat. Na taj način GRU može učiti iz istorijske potrošnje, ali ne vidi vrednost koju treba da predvidi. Isto važi za `Global_reactive_power` i `Global_intensity`: koriste se samo kao istorijske vrednosti iz prethodnih sati u sekvenci, ne kao merenja trenutnog sata.

GRU koristi sledeće jednačine:

```
z_t = σ(W_z · x_t + U_z · h_(t-1) + b_z)
r_t = σ(W_r · x_t + U_r · h_(t-1) + b_r)
h̃_t = tanh(W_h · x_t + r_t * U_h · h_(t-1) + b_h)
h_t = z_t * h_(t-1) + (1 - z_t) * h̃_t
```

z_t je update gate, r_t je reset gate, $\tilde{h}_t$ je kandidat za novo skriveno stanje, a h_t je novo skriveno stanje.

Trening:

- skaliranje se fituje samo nad train skupom,
- validation skup se koristi za early stopping i izbor hiperparametara,
- test skup se koristi za finalnu evaluaciju,
- loss funkcija je MSELoss,
- optimizator je Adam.

Prostor hiperparametara:

Parametar	Vrednosti	Značenje
sequence_length	12, 24, 48	broj prethodnih sati u jednoj sekvenci
batch_size	64	broj sekvenci u jednom trening batch-u
hidden_size	32, 64, 128	veličina skrivene memorije GRU sloja
num_layers	1, 2	broj naslaganih GRU slojeva
dropout	0.0, 0.2, 0.3	regularizacija između slojeva
learning_rate	0.001, 0.0005	brzina učenja optimizer-a
max_epochs	80	maksimalan broj epoha
patience	10	broj epoha bez poboljšanja pre early stopping-a

Ukupno je trenirano **72 GRU modela**. Grid prostor ima 108 mogućih kombinacija, ali se konfiguracije sa `num_layers = 1` i `dropout > 0` preskaču, jer PyTorch dropout u rekurentnom sloju ima efekat samo kada postoji više slojeva.

Najbolja GRU konfiguracija:

Model	seq	hidden	layers	dropout	lr	Validation MAE	Test MAE	Test RMSE	Test WAPE
gru_seq24_h64_l1_d0p0_lr0p0005_bs64	24	64	1	0.0	0.0005	0.3714	0.3291	0.4791	34.3047

Output:

```
models/gru/{model_name}.pt
data/predictions/gru_predictions/
data/logs/gru_logs/
data/logs/compare_logs/baseline_gru_compare_report.csv
data/graphs/gru_graphs/
data/graphs/baseline_gru_compare_graphs/
```

11. LSTM Model

LSTM (Long Short-Term Memory) model se nalazi u `src/step08_lstm_model.py`.

Takođe je rekurentna neuronska mreža, ali za razliku od GRU modela koristi memorijsku ćeliju. To joj omogućava da zadrži informacije kroz duže vremenske zavisnosti.

Priprema podataka, formiranje sekvenci, skaliranje, izbor hiperparametara i evaluacija urađeni su po istom principu kao kod GRU modela.

LSTM koristi isti skup ulaznih feature-a kao GRU i isti princip:

```
prethodnih sequence_length sati -> predikcija trenutnog target sata
```

Hiperparametri:

Parametar	Vrednosti	Značenje
sequence_length	12, 24, 48	broj prethodnih sati u jednoj sekvenci
batch_size	64	broj sekvenci u jednom trening batch-u
hidden_size	32, 64, 128	veličina skrivene memorije LSTM sloja
num_layers	1, 2	broj naslaganih LSTM slojeva
dropout	0.0, 0.2, 0.3	regularizacija između slojeva
learning_rate	0.001, 0.0005	brzina učenja optimizer-a
max_epochs	80	maksimalan broj epoha
patience	10	broj epoha bez poboljšanja pre early stopping-a

Ukupno je trenirano **72 LSTM modela**, kao i kod GRU modela.

Najbolja LSTM konfiguracija:

Model	seq	hidden	layers	dropout	lr	Validation MAE	Test MAE	Test RMSE	Test WAPE
lstm_seq48_h64_l2_d0p2_lr0p0005_bs64	48	64	2	0.2	0.0005	0.3704	0.3267	0.4826	33.9633

Output:

```
models/lstm/{model_name}.pt
data/predictions/lstm_predictions/
data/logs/lstm_logs/
data/logs/compare_logs/baseline_lstm_compare_report.csv
data/graphs/lstm_graphs/
data/graphs/baseline_lstm_compare_graphs/
```

12. Random Forest Regressor

Random Forest model se nalazi u `src/step10_tree_models.py`.

Random Forest je ansambl metoda zasnovana na većem broju decision tree modela. Svako stablo se trenira nezavisno, a konačna regresiona predikcija dobija se kao prosek predikcija svih stabala.

U ovom projektu koristi se `sklearn.ensemble.RandomForestRegressor`.

Kod regresionog stabla svaki unutrašnji čvor sadrži jedan uslov nad jednim feature-om, na primer:

```
lag_1h <= 0.72
```


Stablo bira split koji najviše smanjuje varijansu, odnosno MSE unutar dobijenih podskupova. Za skup S :

$$\text{MSE}(S) = (1 / n) * \sum (y_i - \bar{y})^2$$

Za podelu na levi i desni podskup:

$$\begin{aligned} \text{MSE_after} = & (|S_{\text{left}}| / |S|) * \text{MSE}(S_{\text{left}}) \\ & + (|S_{\text{right}}| / |S|) * \text{MSE}(S_{\text{right}}) \end{aligned}$$

Dobitak splita je:

$$\text{Gain} = \text{MSE}(S) - \text{MSE_after}$$

Stablo bira split sa najvećim smanjenjem greške. Kada novi red stigne do leaf-a, predikcija tog stabla je srednja vrednost targeta trening redova koji su završili u tom leaf-u.

Specifično za implementaciju u projektu:

- `bootstrap=True` je default vrednost u sklearn `RandomForestRegressor` , pa svako stablo dobija bootstrap uzorak redova,
- `max_features` nije posebno podešen, pa je default `max_features=1.0` , odnosno pri splitu se gledaju svi dostupni feature-i,
- nasumičnost potiče pre svega od bootstrap uzorkovanja redova,
- stabla se treniraju greedy izborom najboljih splitova.

Hiperparametri:

Parametar	Vrednosti	Značenje
<code>n_estimators</code>	100, 200	broj stabala u ansamblu
<code>max_depth</code>	10, 20, None	maksimalna dubina stabla
<code>min_samples_leaf</code>	1, 5	minimum redova u jednom leaf-u

Ukupno je trenirano **12 Random Forest modela**, odnosno sve kombinacije iz navedenog prostora hiperparametara.

Najbolja Random Forest konfiguracija:

Model	<code>n_estimators</code>	<code>max_depth</code>	<code>min_samples_leaf</code>	Validation MAE	Test MAE	Test RMSE	Test WAPE
<code>random_forest_n100_depth10_leaf1</code>	100	10	1	0.3737	0.3299	0.4744	34.4243

Najvažniji feature-i za najbolji Random Forest model:

Feature	Importance
<code>lag_1h</code>	0.732366
<code>hour</code>	0.089461
<code>lag_168h</code>	0.057835
<code>lag_24h</code>	0.043082
<code>rolling_mean_24h</code>	0.026531
<code>rolling_std_7d</code>	0.026139

Output:

```
models/random_forest/{model_name}.pkl
data/predictions/random_forest_predictions/
data/logs/random_forest_logs/
data/logs/compare_logs/baseline_random_forest_compare_report.csv
data/graphs/random_forest_graphs/
data/graphs/baseline_random_forest_compare_graphs/
```

Napomena: folder `models/random_forest/` dodat je u `.gitignore` zato što su Random Forest `.pkl` fajlovi veliki. Veličina je posledica čuvanja velikog broja stabala, njihovih čvorova, pragova, vrednosti i strukture.

13. Gradient Boosting Regressor

Gradient Boosting model se nalazi u `src/step10_tree_models.py`, zajedno sa Random Forest modelom.

U ovom projektu koristi se `sklearn.ensemble.GradientBoostingRegressor`.

Gradient Boosting je ansambl metoda u kojoj se stabla treniraju sekvencijalno. Za razliku od Random Forest-a, stabla nisu nezavisna. Svako sledeće stablo uči preostalu grešku prethodno izgrađenog modela.

Za squared error loss početna predikcija je srednja vrednost targeta:

$$F_0(x) = \text{mean}(y)$$

Zatim se računaju reziduali:

$$r_m = y - F_{(m-1)}(x)$$

Novo stablo `h_m(x)` trenira se da predvidi te rezidualne. Ukupna predikcija se zatim ažurira:

$$F_m(x) = F_{(m-1)}(x) + \text{learning_rate} * h_m(x)$$

Za squared error rezidual je istovremeno i negativni gradijent loss funkcije po trenutnoj predikciji. Zbog toga se ovaj postupak naziva gradient boosting.

Specifično za implementaciju u projektu:

- `subsample` nije posebno podešen, pa je default `subsample=1.0` (svako boosting stablo koristi ceo trening skup),
- `max_features` nije posebno podešen, pa se pri splitovima koriste svi dostupni feature-i,
- stabla se dodaju redom i svako novo stablo popravljajući preostalu grešku modela,
- `learning_rate` kontroliše koliko jako jedno novo stablo sme da promeni prethodnu ukupnu predikciju.

Hiperparametri:

Parametar	Vrednosti	Značenje
<code>n_estimators</code>	100, 200	broj sekvencijalnih boosting stabala
<code>learning_rate</code>	0.05, 0.1	veličina korekcije svakog stabla
<code>max_depth</code>	2, 3, 5	maksimalna dubina pojedinačnih stabala

Ukupno je trenirano **12 Gradient Boosting modela**.

Najbolja Gradient Boosting konfiguracija:

Model	n_estimators	learning_rate	max_depth	Validation MAE	Test MAE	Test RMSE	Test WAPE
gradient_boosting_n200_lr0p05_depth5	200	0.05	5	0.3715	0.3269	0.4693	34.1121

Najvažniji feature-i za najbolji Gradient Boosting model:

Feature	Importance
lag_1h	0.758211
hour	0.099021
lag_168h	0.049915
lag_24h	0.035872
rolling_std_7d	0.019620
rolling_mean_24h	0.016929

Output:

```
models/gradient_boosting/{model_name}.pkl
data/predictions/gradient_boosting_predictions/
data/logs/gradient_boosting_logs/
data/logs/compare_logs/baseline_gradient_boosting_compare_report.csv
data/graphs/gradient_boosting_graphs/
data/graphs/baseline_gradient_boosting_compare_graphs/
```

14. Razlika Između Sekvencijalnih i Tree Modela

GRU i LSTM modeli koriste sekvence. Za jednu predikciju posmatraju prethodnih `sequence_length` sati i iz tog prozora uče vremenski obrazac.

Tree modeli ne dobijaju sekvencu u obliku 3D tenzora. Oni dobijaju jedan red tabele, ali taj red već sadrži istorijske informacije kroz:

- `lag_1h` ,
- `lag_24h` ,
- `lag_168h` ,
- `rolling_mean_24h` ,
- `rolling_std_7d` .

Zbog toga tree modeli mogu da koriste prošlost bez eksplicitnog sekvencijalnog ulaza. Razlika je u načinu predstavljanja podataka:

Pristup	Ulaz	Ideja
GRU/LSTM	sekvenca prethodnih sati	model sam uči zavisnosti kroz vreme
RF/GB	jedan red sa lag i rolling feature-ima	istorija je već sažeta u kolone

Razlikuje se i način uklanjanja kolona:

Modeli	Izbačeno iz ulaza	Razlog
GRU/LSTM	<code>datetime</code> , <code>Global_active_power</code>	<code>datetime</code> nije direktan numerički ulaz, a <code>Global_active_power</code> je ekvivalent targeta
RF/GB	<code>datetime</code> , <code>Energy_kWh</code> , <code>Global_active_power</code> , <code>Global_reactive_power</code> , <code>Global_intensity</code>	tree model dobija jedan red trenutnog sata, pa se izbacuju target i sva direktna merenja iz tog sata

Kod GRU/LSTM modela `Energy_kWh` , `Global_reactive_power` i `Global_intensity` mogu postojati u prethodnim satima sekvence. To nije curenje podataka, jer su te vrednosti istorijski poznate pre sata koji se predviđa. Kod RF/GB modela isti red predstavlja trenutni sat, pa bi korišćenje tih kolona značilo da model vidi informacije koje u realnoj predikciji ne bi bile unapred dostupne.

15. Poređenje Rezultata

Poređenje najboljih modela na test skupu:

Model	Najbolja konfiguracija	Test MAE	Test RMSE	Test WAPE
Baseline	<code>lag_24h</code>	0.5123	0.7611	53.4677
GRU	<code>gru_seq24_h64_l1_d0p0_lr0p0005_bs64</code>	0.3291	0.4791	34.3047
LSTM	<code>lstm_seq48_h64_l2_d0p2_lr0p0005_bs64</code>	0.3267	0.4826	33.9633
Random Forest	<code>random_forest_n100_depth10_leaf1</code>	0.3299	0.4744	34.4243
Gradient Boosting	<code>gradient_boosting_n200_lr0p05_depth5</code>	0.3269	0.4693	34.1121

Svi napredniji modeli značajno poboljšavaju rezultate u odnosu na baseline. Razlike između najboljih GRU, LSTM, Random Forest i Gradient Boosting modela su male, što pokazuje da više različitih pristupa uspešno koristi vremenske i istorijske obrasce potrošnje.

16. Vizualizacije

Grafici su organizovani po modelima i tipovima poređenja:

```
data/graphs/baseline_graphs/
data/graphs/gru_graphs/
data/graphs/lstm_graphs/
data/graphs/random_forest_graphs/
data/graphs/gradient_boosting_graphs/
data/graphs/baseline_gru_compare_graphs/
data/graphs/baseline_lstm_compare_graphs/
data/graphs/baseline_random_forest_compare_graphs/
data/graphs/baseline_gradient_boosting_compare_graphs/
data/graphs/all_models_compare_graphs/
```

Individualni grafici prikazuju ponašanje pojedinačnih modela. Uporedni grafici sa baseline modelom prikazuju koliko najbolji model jedne porodice poboljšava rezultat u odnosu na `lag_24h` pristup.

Folder `all_models_compare_graphs` sadrži objedinjene grafike za:

- baseline,
- najbolji GRU,
- najbolji LSTM,

- najbolji Random Forest,
- najbolji Gradient Boosting.

Najvažniji objedinjeni prikazi su:

- poređenje predikcija za najbolji dan u test skupu,
- MAE po satu za sve najbolje modele,
- WAPE po satu za sve najbolje modele,
- summary stubičasti grafici za MAE i RMSE.

Detaljniji interaktivni pregled grafika i tabela planiran je u `docs/powerpredict_walkthrough.ipynb`.

17. Organizacija Izlaznih Fajlova

Glavni izlazi projekta:

Tip izlaza	Lokacija
Obrađeni podaci	<code>data/processed/</code>
Train/validation/test split	<code>data/processed/split/</code>
Logovi i metrike	<code>data/logs/</code>
Predikcije	<code>data/predictions/</code>
Grafici	<code>data/graphs/</code>
PyTorch modeli	<code>models/gru/</code> , <code>models/lstm/</code>
scikit-learn modeli	<code>models/random_forest/</code> , <code>models/gradient_boosting/</code>

PyTorch modeli se čuvaju kao `.pt` fajlovi. Scikit-learn modeli se čuvaju kao `.pkl` fajlovi pomoću `pickle` mehanizma.

18. Zaključak

Baseline `lag_24h` daje korisnu referentnu tačku, ali ima vidljive slabosti u satima u kojima se potrošnja ne ponavlja stabilno iz dana u dan. GRU i LSTM koriste sekvence prethodnih sati, dok Random Forest i Gradient Boosting koriste tabularne lag i rolling feature-e. Sva četiri naprednija modela značajno smanjuju grešku u odnosu na baseline.

Najbolje konfiguracije sva četiri modela, kao i različite konfiguracije istih arhitektura daju vrlo slične rezultate (do nivoa druge i treće decimale). Ovo ukazuje na to da su kvalitetan feature engineering i pravilna vremenska podela podataka jednako važni kao i izbor same arhitekture modela. Dalje značajno poboljšanje verovatno ne bi došlo samo zamenom modela već uvođenjem dodatnih kontekstualnih podataka kao što su informacije o godišnjim odmorima, broju članova domaćinstva, njihovim dnevnim obavezama, vremenskim uslovima, korišćenju većih kućnih uređaja itd. Takvi podaci bi modelima dali dodatno objašnjenje za promene potrošnje koje se ne mogu potpuno zaključiti samo iz istorijskih vrednosti potrošnje.